

Taller de Programación
Estadística Descriptiva en Python

En cuanto al manejo y análisis de estructuras de datos se utilizará, entre otras: la librería "Pandas"

La misma permite trabajar con diferentes estructuras de datos:

*Series: una dimensión.

*DataFrame: tablas de dos dimensiones.

*Panel: Cubos de tres dimensiones.

Para la creación de gráficos se utilizará el módulo "pyplot" de la librería "matplotlib" y la librería seaborn

También se usará "numpy", especializada en cálculo numérico y análisis de datos

Paso 1- Se importa las librerías a utilizar

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sb
```

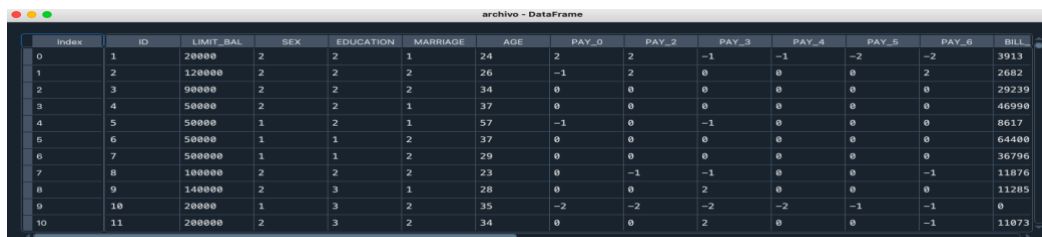
```
import numpy as np
```

Paso 2- Se trae un conjunto de datos xls desde la web

```
site_url = 'https://archive.ics.uci.edu/ml/machine-learning-  
databases/00350/default%20of%20credit%20card%20clients.xls'
```

#Abrimos el archivo como un DataFrame, y le asignamos un nombre

```
archivo = pd.read_excel(site_url, header = 1)
```



Index	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	PAY_6	BILL
0	1	20000	2	2	1	24	2	2	-1	-1	-2	-2	3913
1	2	120000	2	2	2	26	-1	2	0	0	0	2	2682
2	3	90000	2	2	2	34	0	0	0	0	0	0	29239
3	4	50000	2	2	1	37	0	0	0	0	0	0	46990
4	5	50000	1	2	1	57	-1	0	-1	0	0	0	8617
5	6	50000	1	1	2	37	0	0	0	0	0	0	64480
6	7	50000	1	1	2	29	0	0	0	0	0	0	36796
7	8	100000	2	2	2	23	0	-1	-1	0	0	-1	11876
8	9	140000	2	3	1	28	0	0	2	0	0	0	11285
9	10	20000	1	3	2	35	-2	-2	-2	-2	-1	-1	0
10	11	200000	2	3	2	34	0	0	2	0	0	-1	11073

#Hay diferentes formas de leer datos en Pandas:

#pd.read_csv()

#pd.read_excel()

#pd.read_html()

#pd.read_json()

#pd.read_sql()

#pd.read_sas()

#pd.read_pickle()

#Atributos de un DataFrame

#Se puede ver diferentes atributos con las siguientes funciones:

archivo.info()

```
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                    30000 non-null  int64
1   LIMIT_BAL                             30000 non-null  int64
2   SEX                                    30000 non-null  int64
3   EDUCATION                             30000 non-null  int64
4   MARRIAGE                              30000 non-null  int64
5   AGE                                    30000 non-null  int64
6   PAY_0                                 30000 non-null  int64
7   PAY_2                                 30000 non-null  int64
8   PAY_3                                 30000 non-null  int64
9   PAY_4                                 30000 non-null  int64
10  PAY_5                                 30000 non-null  int64
11  PAY_6                                 30000 non-null  int64
12  BILL_AMT1                             30000 non-null  int64
13  BILL_AMT2                             30000 non-null  int64
14  BILL_AMT3                             30000 non-null  int64
15  BILL_AMT4                             30000 non-null  int64
16  BILL_AMT5                             30000 non-null  int64
17  BILL_AMT6                             30000 non-null  int64
18  PAY_AMT1                              30000 non-null  int64
19  PAY_AMT2                              30000 non-null  int64
20  PAY_AMT3                              30000 non-null  int64
21  PAY_AMT4                              30000 non-null  int64
22  PAY_AMT5                              30000 non-null  int64
23  PAY_AMT6                              30000 non-null  int64
24  default payment next month            30000 non-null  int64
dtypes: int64(25)
memory usage: 5.7 MB
```

#devuelve: número de filas, número y tipo de columnas, índices, memoria usada.

archivo.shape

Out[11]: (30000, 25)

#devuelve dimensionalidad

archivo.size

Out[12]: 750000

#devuelve el número de elementos.

archivo.columns

Out[13]:

```
Index(['ID', 'LIMIT_BAL', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_0',  
      'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1', 'BILL_AMT2',  
      'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1',  
      'PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6',  
      'default payment next month'], dtype='object')
```

#devuelve una lista con los nombres de las columnas.

archivo.index

Out[14]: RangeIndex(start=0, stop=30000, step=1)

#devuelve una lista con los nombres de las filas

archivo.dtypes

```
Out[15]:  
ID                int64  
LIMIT_BAL        int64  
SEX              int64  
EDUCATION        int64  
MARRIAGE         int64  
AGE             int64  
PAY_0           int64  
PAY_2           int64  
PAY_3           int64  
PAY_4           int64  
PAY_5           int64  
PAY_6           int64  
BILL_AMT1       int64  
BILL_AMT2       int64  
BILL_AMT3       int64  
BILL_AMT4       int64  
BILL_AMT5       int64  
BILL_AMT6       int64  
PAY_AMT1        int64  
PAY_AMT2        int64  
PAY_AMT3        int64  
PAY_AMT4        int64  
PAY_AMT5        int64  
PAY_AMT6        int64  
default payment next month  int64  
dtype: object
```

#devuelve los tipos de datos de las columnas

archivo.tail(3)

#devuelve las últimas n filas

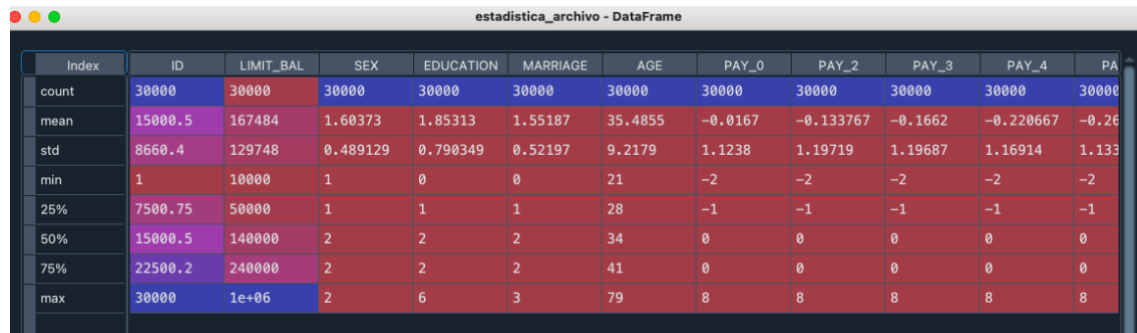
archivo.head(5)

#devuelve las primeras n filas

#Función "DataFrame.describe()" en Pandas:

#Devuelve los datos estadísticos de un archivo DataFrame

```
estadistica_archivo=archivo.describe()
```



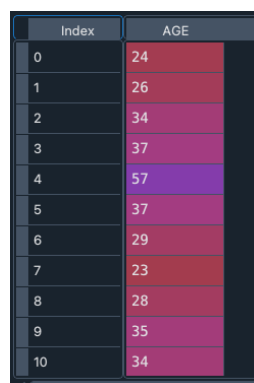
Index	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PA
count	30000	30000	30000	30000	30000	30000	30000	30000	30000	30000	30000
mean	15000.5	167484	1.60373	1.85313	1.55187	35.4855	-0.0167	-0.133767	-0.1662	-0.220667	-0.26
std	8660.4	129748	0.489129	0.790349	0.52197	9.2179	1.1238	1.19719	1.19687	1.16914	1.133
min	1	10000	1	0	0	21	-2	-2	-2	-2	-2
25%	7500.75	50000	1	1	1	28	-1	-1	-1	-1	-1
50%	15000.5	140000	2	2	2	34	0	0	0	0	0
75%	22500.2	240000	2	2	2	41	0	0	0	0	0
max	30000	1e+06	2	6	3	79	8	8	8	8	8

#devuelve: cantidad de elementos por columna, media, desvío estandar,

#percentiles 25,50,75, valores máximos y mínimos

#En el caso de requerir datos de una columna en particular

```
edades = archivo["AGE"]
```



Index	AGE
0	24
1	26
2	34
3	37
4	57
5	37
6	29
7	23
8	28
9	35
10	34

```
edades.describe()
```

```
Out[20]:
count    30000.000000
mean      35.485500
std        9.217904
min       21.000000
25%       28.000000
50%       34.000000
75%       41.000000
max       79.000000
Name: AGE, dtype: float64
```

#también podemos obtener la información por separado:

```
edades.count()
```

#devuelve el número de elementos

```
Out[24]: 30000
```

```
edades.sum()
```

```
#devuelve la suma de los datos
```

```
Out[25]: 1064565
```

```
edades.cumsum()
```

```
#devuelve la suma acumulada de los datos
```

```
Out[26]:
```

```
0      24
1      50
2      84
3     121
4     178
```

```
29995  1064398
```

```
29996  1064441
```

```
29997  1064478
```

```
29998  1064519
```

```
29999  1064565
```

```
Name: AGE, Length: 30000, dtype: int64
```

```
edades.min()
```

```
#devuelve el dato mínimo
```

```
Out[27]: 21
```

```
edades.max()
```

```
#devuelve el dato máximo
```

```
Out[28]: 79
```

```
edades.mean()
```

```
#devuelve la media de los datos
```

```
Out[29]: 35.4855
```

```
edades.var()
```

```
#devuelve la varianza
```

```
Out[30]: 84.96975540851363
```

```
edades.std()
```

```
#devuelve el desvío estandar
```

```
Out[31]: 9.217904068090188
```

```
#Se puede guardar el resumen de estadísticas en un csv
```

```
estadistica_archivo.to_csv('estadistica.csv', sep = ";")
```

```
#corroboramos que los datos se guardaron correctamente leyendo el archivo
```

```
resumen_archivo = pd.read_csv('estadistica.csv', sep=";")
```

```
resumen_archivo
```

Gráficos

Gráficos de barras a partir del archivo en estudio

Paso 1- seleccionamos los datos a graficar

```
#(si queremos graficar la cantidad de casos por categoría)
```

```
#armamos la serie a graficar
```

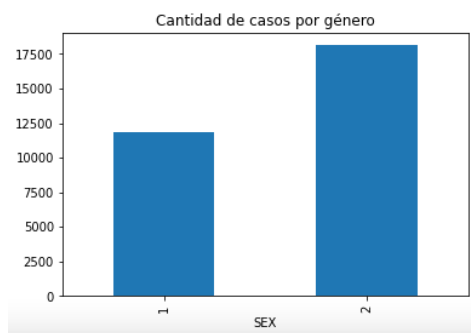
```
genero = archivo.groupby('SEX').count()['ID']
```

genero - Series	
SEX	ID
1	11888
2	18112

Paso 2- realizamos un gráfico de barras

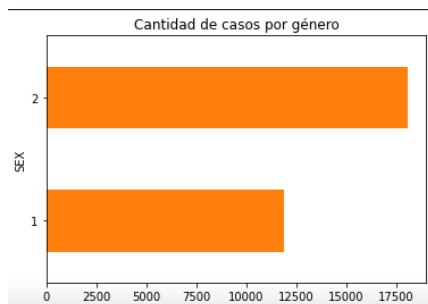
#vertical:

```
genero.plot(kind='bar', title="Cantidad de casos por género", color="tab:blue")
```



#horizontal:

```
genero.plot(kind='barh', title="Cantidad de casos por género", color="tab:orange")
```



#otro ejemplo:

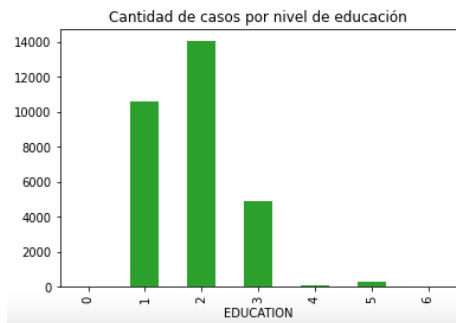
Graficamos la cantidad de casos según nivel de educación:

```
educacion = archivo.groupby('EDUCATION').count()['ID']
```

EDUCATION	ID
0	14
1	10585
2	14030
3	4917
4	123
5	280
6	51

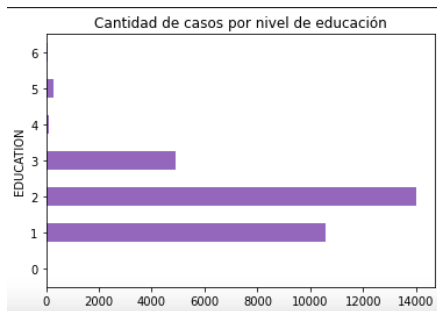
```
#vertical
```

```
educacion.plot(kind = 'bar', title="Cantidad de casos por nivel de educación",  
color="tab:green")
```



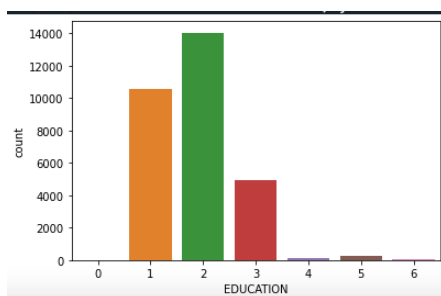
```
#horizontal
```

```
educacion.plot(kind = 'barh', title="Cantidad de casos por nivel de educación",  
color="tab:purple")
```



```
#usando Seaborn
```

```
sb.countplot(x='EDUCATION', data=archivo)
```



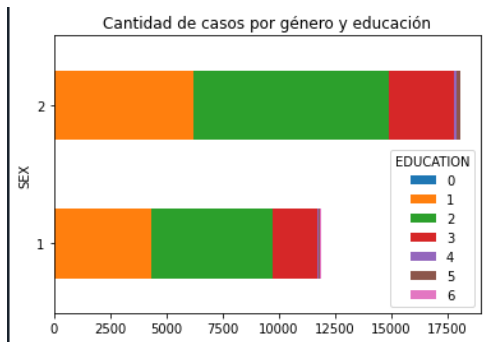
En el caso de combinar 2 o más variables:

```
#cantidad de casos según género y educación:
```

```
genero_educacion= archivo.groupby(["SEX","EDUCATION"])["ID"].count().unstack()
```

	SEX	0	1	2	3	4	5	6
1		8	4354	5374	1990	42	95	25
2		6	6231	8656	2927	81	185	26

genero_educacion.plot(kind="barh",stacked=True, title="Cantidad de casos por género y educación")



genero_educacion2= archivo.groupby(["EDUCATION","SEX"])["ID"].count().unstack()

EDUCATION	1	2
0	8	6
1	4354	6231
2	5374	8656
3	1990	2927
4	42	81
5	95	185
6	25	26

genero_educacion2.plot(kind="bar",stacked=True, title="Cantidad de casos por género y educación")

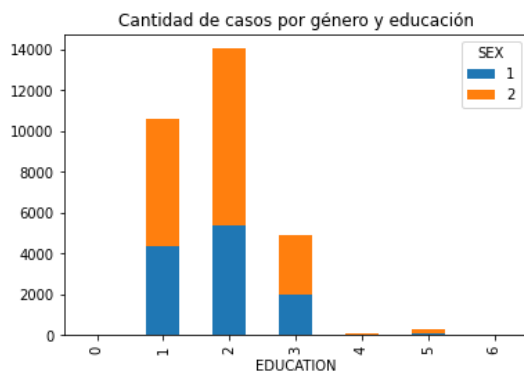
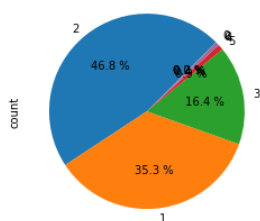


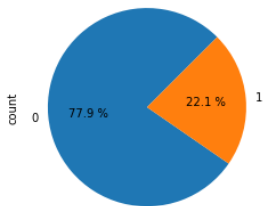
Gráfico de torta

#grafica un diagrama de sectores con las frecuencias de la variable seleccionada

archivo['EDUCATION'].value_counts().plot.pie(startangle=45,autopct='%1f %%%')

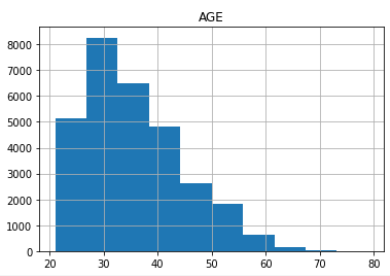


`archivo["default payment next month"].value_counts().plot.pie(startangle=45,autopct='%1f%%')`

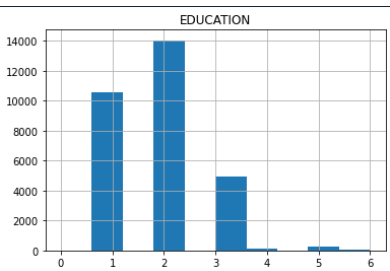


Histogramas

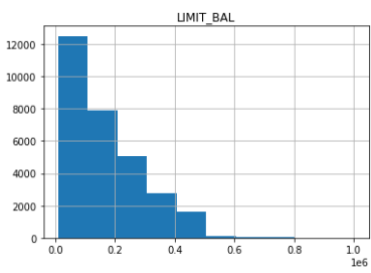
`archivo.hist("AGE")`



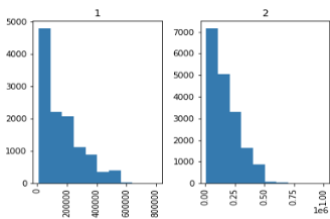
`archivo.hist("EDUCATION")`



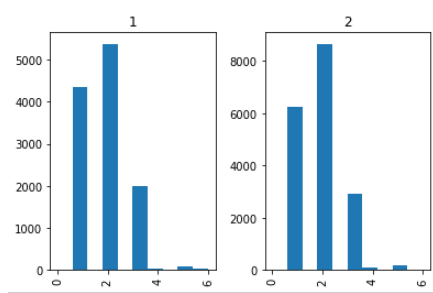
`archivo.hist('LIMIT_BAL')`



`archivo.hist("LIMIT_BAL", "SEX")`

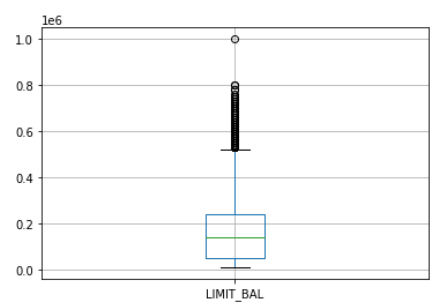


```
archivo.hist("EDUCATION", "SEX")
```

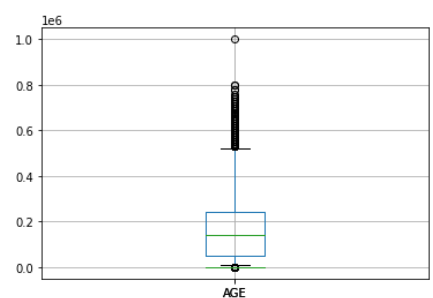


Boxplot

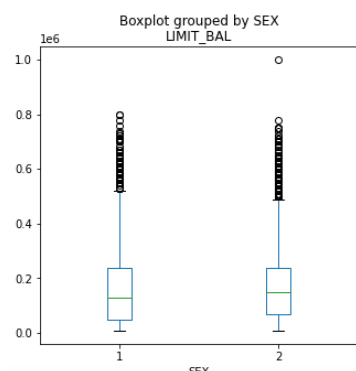
```
archivo.boxplot('LIMIT_BAL')
```



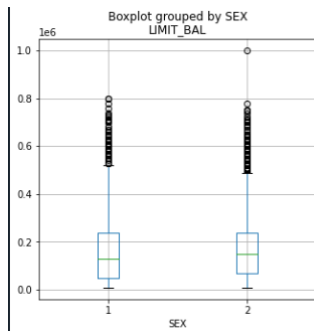
```
archivo.boxplot("AGE")
```



```
archivo.boxplot('LIMIT_BAL', 'SEX', figsize=(5, 5), grid=False)
```



```
archivo.boxplot('LIMIT_BAL', 'SEX', figsize=(5, 5), grid=True)
```



#usando Seaborn:

```
sb.boxplot(data=archivo['LIMIT_BAL'])
```

```
plt.ticklabel_format(style='plain', axis='y')
```

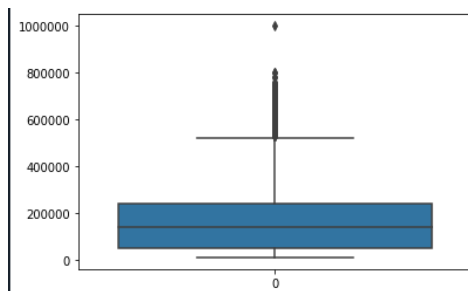
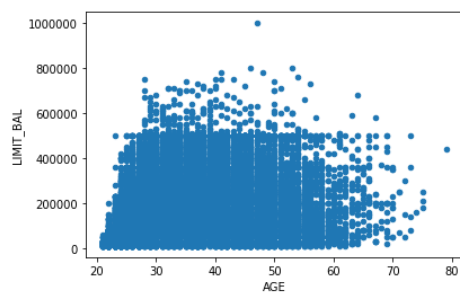


Diagrama de dispersión

```
archivo.plot(kind = 'scatter', x='AGE', y='LIMIT_BAL')
```

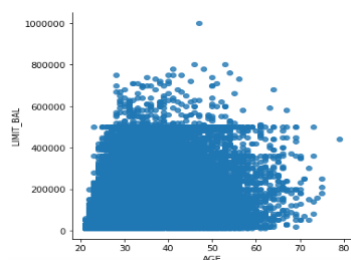
```
plt.ticklabel_format(style='plain', axis='y')
```



#usando Seaborn:

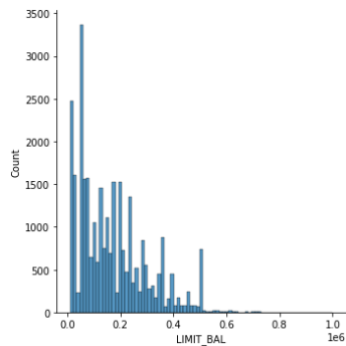
```
sb.lmplot(x='AGE', y='LIMIT_BAL', data=archivo, fit_reg=False)
```

```
plt.ticklabel_format(style='plain', axis='y')
```



densidad de una variable

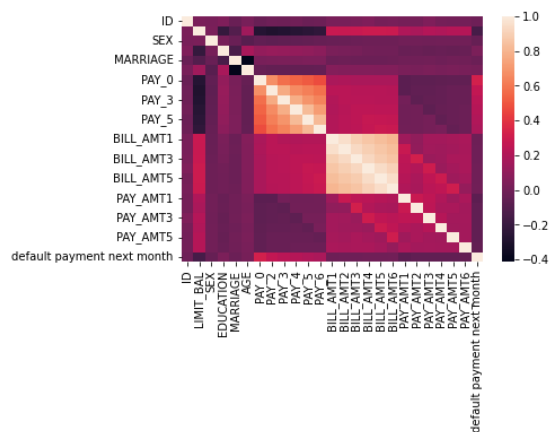
```
sb.displot(archivo['LIMIT_BAL'])
```



#Matriz de correlación

```
corr = archivo.corr()
```

```
sb.heatmap(corr)
```



Otro ejemplo

```
url = "https://raw.githubusercontent.com/sanchezgaviermatias/Curso_Python-/master/2%20-%20Pandas/Pokemon.csv"
```

```
archivo_2 = pd.read_csv(url)
```

archivo_2 - DataFrame											
Index	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80
4	4	Charmander	Fire	nan	309	39	52	43	60	50	65
5	5	Charmeleon	Fire	nan	405	58	64	58	80	65	80
6	6	Charizard	Fire	Flying	534	78	84	78	109	85	100
7	6	CharizardMega Charizard X	Fire	Dragon	634	78	130	111	130	85	100
8	6	CharizardMega Charizard Y	Fire	Flying	634	78	104	78	159	115	100
9	7	Squirtle	Water	nan	314	44	48	65	50	64	43
10	8	Wartortle	Water	nan	405	59	63	80	65	80	58

#Miramos algunos atributos:

archivo_2.info()

```
In [56]:
...: archivo_2.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  -
0    #           800 non-null   int64
1    Name        800 non-null   object
2    Type 1      800 non-null   object
3    Type 2      414 non-null   object
4    Total       800 non-null   int64
5    HP          800 non-null   int64
6    Attack      800 non-null   int64
7    Defense     800 non-null   int64
8    Sp. Atk     800 non-null   int64
9    Sp. Def     800 non-null   int64
10   Speed       800 non-null   int64
11   Generation  800 non-null   int64
12   Legendary   800 non-null   bool
dtypes: bool(1), int64(9), object(3)
memory usage: 75.9+ KB
```

archivo_2.shape

Out[57]: (800, 13)

type(archivo_2)

Out[58]: pandas.core.frame.DataFrame

archivo_2.columns

Out[59]:

```
Index(['#', 'Name', 'Type 1', 'Type 2', 'Total', 'HP', 'Attack', 'Defense',
      'Sp. Atk', 'Sp. Def', 'Speed', 'Generation', 'Legendary'],
      dtype='object')
```

archivo_2.dtypes

Out[60]:

```
#           int64
Name        object
Type 1      object
Type 2      object
Total       int64
HP          int64
```

```

Attack      int64
Defense     int64
Sp. Atk     int64
Sp. Def     int64
Speed       int64
Generation  int64
Legendary   bool
dtype: object

```

```
type(archivo_2["Type 1"])
```

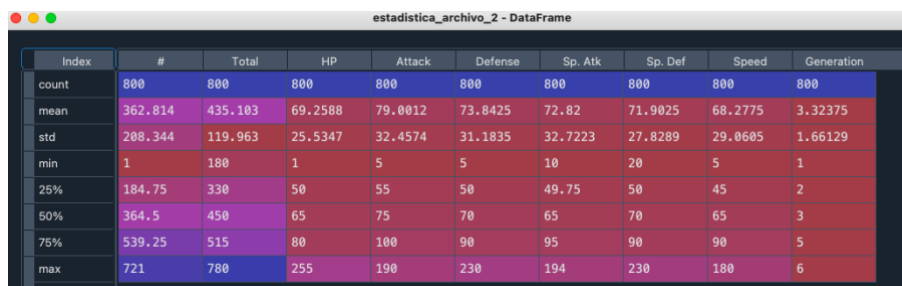
```
Out[61]: pandas.core.series.Series
```

```
archivo_2.index
```

```
Out[62]: RangeIndex(start=0, stop=800, step=1)
```

Vemos sus estadísticas:

```
estadistica_archivo_2=archivo_2.describe()
```



	Index	#	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation
count		800	800	800	800	800	800	800	800	800
mean		362.814	435.183	69.2588	79.0012	73.8425	72.82	71.9025	68.2775	3.32375
std		208.344	119.963	25.5347	32.4574	31.1835	32.7223	27.8289	29.0605	1.66129
min		1	180	1	5	5	10	20	5	1
25%		184.75	330	50	55	50	49.75	50	45	2
50%		364.5	450	65	75	70	65	70	65	3
75%		539.25	515	80	100	90	95	90	90	5
max		721	780	255	190	230	194	230	180	6

#Podemos localizar datos que tengan condiciones

#generamos condiciones

```
condicion = (archivo_2["Type 1"] == "Grass") & ~(archivo_2["HP"]>=80)
```

#generamos un nuevo DataFrame con los datos seleccionados

#la función .loc busca en términos de valores

```
datos_seleccionados=archivo_2.loc[condicion, :]
```

Index	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1
48	43	Oddish	Grass	Poison	320	45	50	55	75	65	30	1
49	44	Gloom	Grass	Poison	395	60	65	70	85	75	40	1
50	45	Vileplume	Grass	Poison	490	75	80	85	110	90	50	1
75	69	Bellsprout	Grass	Poison	300	50	75	35	70	30	40	1
76	70	Weepinbell	Grass	Poison	390	65	90	50	85	45	55	1
110	102	Exeggcute	Grass	Psychic	325	60	40	80	60	45	40	1
122	114	Tangela	Grass	nan	435	65	55	115	100	40	60	1
166	152	Chikorita	Grass	nan	318	45	49	65	49	65	45	2
167	153	Bayleef	Grass	nan	405	60	62	80	63	80	60	2

#cambiamos la condición

```
condicion = (archivo_2["Type 1"].isin(["Grass", "Fire"]))
```

```
datos_seleccionados_2=archivo_2.loc[ condicion, : ]
```

Index	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80
4	4	Charmander	Fire	nan	309	39	52	43	60	50	65
5	5	Charmeleon	Fire	nan	405	58	64	58	80	65	80
6	6	Charizard	Fire	Flying	534	78	84	78	109	85	100
7	6	CharizardMega Charizard X	Fire	Dragon	634	78	130	111	130	85	100
8	6	CharizardMega Charizard Y	Fire	Flying	634	78	104	78	159	115	100
42	37	Vulpix	Fire	nan	299	38	41	40	50	65	65
43	38	Ninetales	Fire	nan	505	73	76	75	81	100	100

#localizamos datos por su posición

#usamos la función .iloc

#seleccionamos las primeras 10 filas y todas las columnas

```
archivo_2.iloc[0:10, :]
```

```
archivo_2.iloc[0:10, :]
```

Out[67]:

	#	Name	Type 1	...	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	...	45	1	False
1	2	Ivysaur	Grass	...	60	1	False
2	3	Venusaur	Grass	...	80	1	False

3	3	VenusaurMega Venusaur	Grass ...	80	1	False
4	4	Charmander	Fire ...	65	1	False
5	5	Charmeleon	Fire ...	80	1	False
6	6	Charizard	Fire ...	100	1	False
7	6	CharizardMega Charizard X	Fire ...	100	1	False
8	6	CharizardMega Charizard Y	Fire ...	100	1	False
9	7	Squirtle	Water ...	43	1	False

[10 rows x 13 columns]

#seleccionamos las primeras 10 (0 a 9)filas y columnas 2,3 y 4

archivo_2.iloc[0:10,1:4]

archivo_2.iloc[0:10,1:4]

Out[68]:

	Name	Type 1	Type 2
0	Bulbasaur	Grass	Poison
1	Ivysaur	Grass	Poison
2	Venusaur	Grass	Poison
3	VenusaurMega Venusaur	Grass	Poison
4	Charmander	Fire	NaN
5	Charmeleon	Fire	NaN
6	Charizard	Fire	Flying
7	CharizardMega Charizard X	Fire	Dragon
8	CharizardMega Charizard Y	Fire	Flying
9	Squirtle	Water	NaN

#seleccionamos distintas columnas y les cambiamos el nombre

```
archivo_3=archivo_2[["Name","Speed","Generation"]]
```

```
archivo_3.columns=["Nombre","Velocidad","Generación"]
```



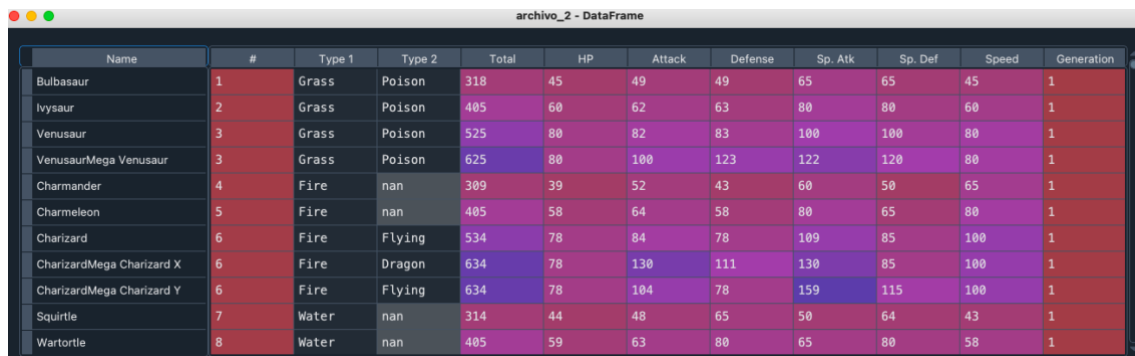
Index	Nombre	Velocidad	Generación
0	Bulbasaur	45	1
1	Ivysaur	60	1
2	Venusaur	80	1
3	VenusaurMega Venusaur	80	1
4	Charmander	65	1
5	Charmeleon	80	1
6	Charizard	100	1
7	CharizardMega Charizard X	100	1
8	CharizardMega Charizard Y	100	1
9	Squirtle	43	1
10	Wartortle	58	1

```
# le indicamos una columna para transformarla en el índice
```

```
# Para que los cambios sean permanentes:
```

```
#usamos inplace=True
```

```
archivo_2.set_index("Name", inplace=True)
```



Name	#	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation
Bulbasaur	1	Grass	Poison	318	45	49	49	65	65	45	1
Ivysaur	2	Grass	Poison	405	60	62	63	80	80	60	1
Venusaur	3	Grass	Poison	525	80	82	83	100	100	80	1
VenusaurMega Venusaur	3	Grass	Poison	625	80	100	123	122	120	80	1
Charmander	4	Fire	nan	309	39	52	43	60	50	65	1
Charmeleon	5	Fire	nan	405	58	64	58	80	65	80	1
Charizard	6	Fire	Flying	534	78	84	78	109	85	100	1
CharizardMega Charizard X	6	Fire	Dragon	634	78	130	111	130	85	100	1
CharizardMega Charizard Y	6	Fire	Flying	634	78	104	78	159	115	100	1
Squirtle	7	Water	nan	314	44	48	65	50	64	43	1
Wartortle	8	Water	nan	405	59	63	80	65	80	58	1

```
#Buscamos un dato a través del índice con la función loc .
```

```
archivo_2.loc["Bulbasaur", : ]
```

```
Out[71]:
```

```
#          1
Type 1    Grass
Type 2    Poison
Total      318
HP         45
Attack     49
Defense    49
```

Sp. Atk 65

Sp. Def 65

Speed 45

Generation 1

Legendary False

Name: Bulbasaur, dtype: object

archivo_2.loc[["Bulbasaur","Munna"], :]

Out[72]:

	#	Type 1	Type 2	Total	...	Sp. Def	Speed	Generation	Legendary
Name					...				
Bulbasaur	1	Grass	Poison	318	...	65	45	1	False
Munna	517	Psychic	NaN	292	...	55	24	5	False

[2 rows x 12 columns]

Datos faltantes

#Métodos de resolución de NaN

Hay 3 opciones:

Eliminar Filas con NaN

Eliminar Columna con NaN

Reemplazar NaN con Otros Valores

#Paso 1- vemos donde hay datos faltantes

archivo_2.info()

<class 'pandas.core.frame.DataFrame'>

Index: 800 entries, Bulbasaur to Volcanion

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

---	-----	-----	-----
-----	-------	-------	-------

0	#	800 non-null	int64
1	Type 1	800 non-null	object
2	Type 2	414 non-null	object
3	Total	800 non-null	int64
4	HP	800 non-null	int64
5	Attack	800 non-null	int64
6	Defense	800 non-null	int64
7	Sp. Atk	800 non-null	int64
8	Sp. Def	800 non-null	int64
9	Speed	800 non-null	int64
10	Generation	800 non-null	int64
11	Legendary	800 non-null	bool

dtypes: bool(1), int64(9), object(2)

memory usage: 108.1+ KB

#otra forma de verlo:

archivo_2.isnull().sum()

Out[74]:

#	0
Type 1	0
Type 2	386
Total	0
HP	0
Attack	0

```
Defense      0
Sp. Atk      0
Sp. Def      0
Speed        0
Generation   0
Legendary    0
```

```
dtype: int64
```

```
#muestra que campo tiene datos faltantes y los cuantifica
```

```
# Ordenamos de mayor a menor el detalle anterior
```

```
archivo_2.isna().sum().sort_values(ascending=False)
```

```
Out[75]:
```

```
Type 2      386
```

```
#          0
```

```
Type 1      0
```

```
Total       0
```

```
HP          0
```

```
Attack      0
```

```
Defense     0
```

```
Sp. Atk     0
```

```
Sp. Def     0
```

```
Speed       0
```

```
Generation  0
```

```
Legendary   0
```

```
dtype: int64
```

```
#Paso 2- resolver los datos faltantes
```

#Método 1 =eliminar columna, (no recomendado)

```
archivo_2.drop("Type 2",axis=1).head()
```

Out[76]:

```
      # Type 1 Total ... Speed Generation Legendary
Name
Bulbasaur      1 Grass  318 ...   45      1  False
Ivysaur        2 Grass  405 ...   60      1  False
Venusaur       3 Grass  525 ...   80      1  False
VenusaurMega Venusaur 3 Grass  625 ...   80      1  False
Charmander     4 Fire  309 ...   65      1  False
```

#Método 2 =eliminar Filas, mejor que lo anterior (no recomendado)

```
y = archivo_2.dropna()
```

#notar que se pierden muchos datos:

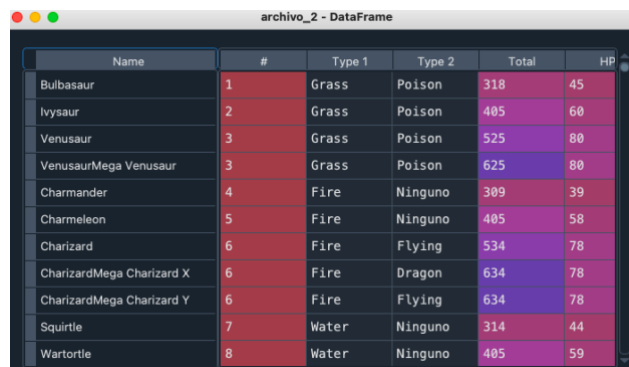
```
print(y.shape, archivo_2.shape)
```

```
(414, 12) (800, 12)
```

#Método 3= Reemplazar los datos faltantes

#se recomienda este!!!!

```
archivo_2["Type 2"]=archivo_2["Type 2"].fillna("Ninguno")
```



Name	#	Type 1	Type 2	Total	HP
Bulbasaur	1	Grass	Poison	318	45
Ivysaur	2	Grass	Poison	405	60
Venusaur	3	Grass	Poison	525	80
VenusaurMega Venusaur	3	Grass	Poison	625	80
Charmander	4	Fire	Ninguno	309	39
Charmeleon	5	Fire	Ninguno	405	58
Charizard	6	Fire	Flying	534	78
CharizardMega Charizard X	6	Fire	Dragon	634	78
CharizardMega Charizard Y	6	Fire	Flying	634	78
Squirtle	7	Water	Ninguno	314	44
Wartortle	8	Water	Ninguno	405	59

#otra forma si fuera variables numéricas:

```
#archivo_2["Type 2"].fillna((archivo_2["Type 2"].mode()), inplace=True)
```

```
#Paso 3- chequeamos resultados
```

```
archivo_2.isnull().sum()
```

```
Out[81]:
```

```
#      0
Type 1  0
Type 2  0
Total   0
HP      0
Attack  0
Defense 0
Sp. Atk 0
Sp. Def 0
Speed   0
Generation 0
Legendary 0
dtype: int64
```

```
## Operaciones a realizar sobre el conjunto de datos:
```

```
#sumar los valores de un campo determinado
```

```
archivo_2["HP"].sum()
```

```
Out[83]: 55407
```

```
#ordenar en referencia a un campo determinado
```

```
archivo_4=archivo_2.sort_values(by='Name')
```

Name	#	Type 1	Type 2	Total	HP
Abomasnow	460	Grass	Ice	494	90
AbomasnowMega Abomasnow	460	Grass	Ice	594	90
Abra	63	Psychic	Ninguno	310	25
Absol	359	Dark	Ninguno	465	65
AbsolMega Absol	359	Dark	Ninguno	565	65
Accelgor	617	Bug	Ninguno	495	80
AegislashBlade Forme	681	Steel	Ghost	520	60
AegislashShield Forme	681	Steel	Ghost	520	60
Aerodactyl	142	Rock	Flying	515	80
AerodactylMega Aerodactyl	142	Rock	Flying	615	80
Aggron	306	Steel	Rock	530	70

#ver la media de los valores de un campo determinado

```
archivo_2["HP"].mean()
```

Out[85]: 69.25875

#ver el desvío estandar de los valores de un campo determinado

```
archivo_2["HP"].std()
```

Out[86]: 25.53466903233207

Operaciones entre Columnas

```
archivo_2["Poder de Ataque"] = archivo_2["HP"]*(1/3) +  
archivo_2["Attack"]*(1/3)+archivo_2["Sp. Atk"]*(1/3)
```

```
archivo_2["Poder de Ataque"] = round(archivo_2["Poder de Ataque"])
```

```
archivo_2["Poder de Defensa"] = archivo_2["HP"]*(1/3) +  
archivo_2["Defense"]*(1/3)+archivo_2["Sp. Def"]*(1/3)
```

```
archivo_2["Poder de Defensa"] = round(archivo_2["Poder de Defensa"])
```



archivo_2 - DataFrame

Name	p. Atk	Sp. Def	Speed	Generation	egendar	Poder de Ataque	Poder de Defensa
Bulbasaur	65	45	1	False	53	53	
Ivysaur	80	60	1	False	67	68	
Venusaur	100	80	1	False	87	88	
VenusaurMega Venusaur	120	80	1	False	101	108	
Charmander	50	65	1	False	50	44	
Charmeleon	65	80	1	False	67	60	
Charizard	85	100	1	False	90	80	
CharizardMega Charizard X	85	100	1	False	113	91	
CharizardMega Charizard Y	115	100	1	False	114	90	
Squirtle	64	43	1	False	47	58	
Wartortle	80	58	1	False	62	73	

```
#Averiguamos el valor Z
```

```
archivo_2["Z_HP"] = (archivo_2["HP"] - archivo_2["HP"].mean()) / archivo_2["HP"].std()
```

```
archivo_2.head()
```

```
Out[89]:
```

Name	#	Type 1	...	Poder de Defenza	Z_HP
Bulbasaur	1	Grass	...	53.0	-0.950032
Ivysaur	2	Grass	...	68.0	-0.362595
Venusaur	3	Grass	...	88.0	0.420654
VenusaurMega Venusaur	3	Grass	...	108.0	0.420654
Charmander	4	Fire	...	44.0	-1.185007

```
#Vemos los valores únicos
```

```
archivo_2["Type 1"].unique()
```

```
Out[90]:
```

```
array(['Grass', 'Fire', 'Water', 'Bug', 'Normal', 'Poison', 'Electric',  
      'Ground', 'Fairy', 'Fighting', 'Psychic', 'Rock', 'Ghost', 'Ice',  
      'Dragon', 'Dark', 'Steel', 'Flying'], dtype=object)
```

```
#los contamos
```

```
archivo_2['Type 1'].nunique()
```

```
Out[93]: 18
```

```
#Vemos la frecuencia Absoluta de cada uno
```

```
archivo_2['Type 1'].value_counts()
```

```
Out[94]:
```

```
Type 1
```

```
Water    112
```

```
Normal    98
```

```
Grass     70
```

```
Bug       69
```

```
Psychic   57
```

```
Fire      52
```

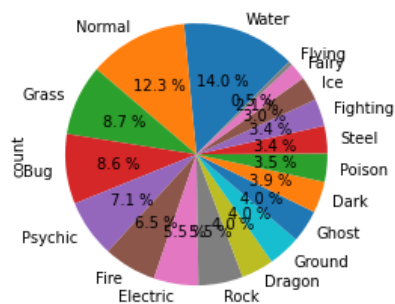
```
Electric  44
```

Rock 44
 Dragon 32
 Ground 32
 Ghost 32
 Dark 31
 Poison 28
 Steel 27
 Fighting 27
 Ice 24
 Fairy 17
 Flying 4

Name: count, dtype: int64

#Realizamos un gráfico de torta y vemos la frecuencia relativa

archivo_2['Type 1'].value_counts().plot.pie(startangle=45,autopct='%.1f %%')



Columnas Condicionales

#creamos una nueva variable a partir de otras

#generamos una columna que clasifique el tiempo de reacción en lenta,mediana,veloz,muy veloz

archivo_2["Speed"].unique()

Out[95]:

array([45, 60, 80, 65, 100, 43, 58, 78, 30, 70, 50, 35, 75,

145, 56, 71, 101, 121, 72, 97, 55, 90, 110, 40, 41, 76,
 85, 20, 25, 95, 120, 115, 105, 150, 15, 130, 42, 67, 140,
 87, 63, 68, 93, 81, 48, 91, 33, 5, 83, 51, 61, 125,
 160, 28, 135, 10, 23, 32, 52, 180, 31, 36, 108, 66, 34,
 39, 112, 74, 84, 82, 102, 92, 47, 46, 86, 77, 127, 113,
 106, 64, 24, 29, 116, 114, 88, 69, 57, 98, 22, 44, 59,
 79, 103, 109, 38, 111, 128, 99, 73, 104, 122, 62, 126, 89,
 49, 118, 54, 123])

```
archivo_2.loc[archivo_2["Speed"] <=50 , 'Reacción'] = "Lenta"
```

```
archivo_2.loc[(51<=archivo_2["Speed"]) & (archivo_2["Speed"] <=100) , 'Reacción'] =  
"Mediana"
```

```
archivo_2.loc[(101<=archivo_2["Speed"]) & (archivo_2["Speed"] <=150) , 'Reacción'] = "Veloz"
```

```
archivo_2.loc[archivo_2["Speed"]>=151 , 'Reacción'] = "Muy Veloz"
```



archivo_2 - DataFrame

Name	lary	loder de Ataqu	oder de Defenz	Z_HP	Reacción
Bulbasaur		53	53	-0.950032	Lenta
Ivysaur		67	68	-0.362595	Mediana
Venusaur		87	88	0.420654	Mediana
VenusaurMega Venusaur		101	108	0.420654	Mediana
Charmander		50	44	-1.18501	Mediana
Charmeleon		67	60	-0.44092	Mediana
Charizard		90	80	0.342329	Mediana
CharizardMega Charizard X		113	91	0.342329	Mediana
CharizardMega Charizard Y		114	90	0.342329	Mediana
Squirtle		47	58	-0.989194	Lenta
Wartortle		62	73	-0.401758	Mediana

Podemos aplicar funciones a las columnas

```
def pokemon_poder(poder):
```

```
    """Imprime los pokemon con un determinado poder seleccionado"""
```

```

poder = (archivo_2["Type 1"].isin([poder]))

datos_selec=archivo_2.loc[ poder, :]

return print(datos_selec)

pokemon_poder("Fire")

```

	# Type 1	...	Z_HP	Reacción
Name		...		
Charmander	4	Fire ...	-1.185007	Mediana
Charmeleon	5	Fire ...	-0.440920	Mediana
Charizard	6	Fire ...	0.342329	Mediana
CharizardMega Charizard X	6	Fire ...	0.342329	Mediana
CharizardMega Charizard Y	6	Fire ...	0.342329	Mediana
Vulpix	37	Fire ...	-1.224169	Mediana
Ninetales	38	Fire ...	0.146516	Mediana
Growlithe	58	Fire ...	-0.558407	Mediana
Arcanine	59	Fire ...	0.812278	Mediana
Ponyta	77	Fire ...	-0.754220	Mediana
Rapidash	78	Fire ...	-0.166783	Veloz
Magmar	126	Fire ...	-0.166783	Mediana
Flareon	136	Fire ...	-0.166783	Mediana
Moltres	146	Fire ...	0.812278	Mediana
Cyndaquil	155	Fire ...	-1.185007	Mediana
Quilava	156	Fire ...	-0.440920	Mediana
Typhlosion	157	Fire ...	0.342329	Mediana
Slugma	218	Fire ...	-1.145844	Lenta
Magcargo	219	Fire ...	-0.754220	Lenta
Magby	240	Fire ...	-0.950032	Mediana
Entei	244	Fire ...	1.791339	Mediana

Ho-oh	250	Fire ...	1.438877	Mediana
Torchic	255	Fire ...	-0.950032	Lenta
Combusken	256	Fire ...	-0.362595	Mediana
Blaziken	257	Fire ...	0.420654	Mediana
BlazikenMega Blaziken	257	Fire ...	0.420654	Mediana
Numel	322	Fire ...	-0.362595	Lenta
Camerupt	323	Fire ...	0.029029	Lenta
CameruptMega Camerupt	323	Fire ...	0.029029	Lenta
Torkoal	324	Fire ...	0.029029	Lenta
Chimchar	390	Fire ...	-0.989194	Mediana
Monferno	391	Fire ...	-0.205945	Mediana
Infernape	392	Fire ...	0.264004	Veloz
Magmortar	467	Fire ...	0.224841	Mediana
Heatran	485	Fire ...	0.851440	Mediana
Tepig	498	Fire ...	-0.166783	Lenta
Pignite	499	Fire ...	0.812278	Mediana
Emboar	500	Fire ...	1.595527	Mediana
Pansear	513	Fire ...	-0.754220	Mediana
Simisear	514	Fire ...	0.224841	Veloz
Darumaka	554	Fire ...	0.029029	Lenta
DarmanitanStandard Mode	555	Fire ...	1.399715	Mediana
DarmanitanZen Mode	555	Fire ...	1.399715	Mediana
Heatmor	631	Fire ...	0.616466	Mediana
Fennekin	653	Fire ...	-1.145844	Mediana
Braixen	654	Fire ...	-0.401758	Mediana
Delphox	655	Fire ...	0.224841	Veloz
Fletchinder	662	Fire ...	-0.284270	Mediana

Talonflame	663	Fire ...	0.342329	Veloz
Litleo	667	Fire ...	-0.284270	Mediana
Pyroar	668	Fire ...	0.655628	Veloz
Volcanion	721	Fire ...	0.420654	Mediana

[52 rows x 16 columns]

pokemon_poder("Grass")

	#	Type 1	...	Z_HP	Reacción
Name					...
Bulbasaur	1	Grass	...	-0.950032	Lenta
Ivysaur	2	Grass	...	-0.362595	Mediana
Venusaur	3	Grass	...	0.420654	Mediana
VenusaurMega Venusaur	3	Grass	...	0.420654	Mediana
Oddish	43	Grass	...	-0.950032	Lenta
	...	...	...	...	...
Chespin	650	Grass	...	-0.519245	Lenta
Quilladin	651	Grass	...	-0.323433	Mediana
Chesnaught	652	Grass	...	0.733953	Mediana
Skiddo	672	Grass	...	-0.127621	Mediana
Gogoat	673	Grass	...	2.104639	Mediana

[70 rows x 16 columns]

Otras operaciones son:

```
print(archivo_2["Speed"].min()) # Minimo
print(archivo_2["Speed"].max()) # Maximo
print(archivo_2["Speed"].count()) # Cantidad
print(archivo_2["Speed"].idxmax()) # El índice del valor máximo
print(archivo_2["Speed"].idxmin()) # El índice del valor mínimo
```

```
print(archivo_2["Speed"].quantile([.25,.5,.75])) # Los quantiles
```

```
print(archivo_2["Speed"].skew()) # Asimetria
```

```
print(archivo_2["Speed"].kurtosis()) # Kurtosis
```

```
In [101]: print(archivo_2["Speed"].min()) # Minimo
...: print(archivo_2["Speed"].max()) # Maximo
...: print(archivo_2["Speed"].count()) # Cantidad
...: print(archivo_2["Speed"].idxmax()) # El índice del valor máximo
...: print(archivo_2["Speed"].idxmin()) # El índice del valor mínimo
...: print(archivo_2["Speed"].quantile([.25,.5,.75])) # Los quantiles
...: print(archivo_2["Speed"].skew()) # Asimetria
...: print(archivo_2["Speed"].kurtosis()) # Kurtosis
5
180
800
DeoxysSpeed Forme
Shuckle
0.25    45.0
0.50    65.0
0.75    90.0
Name: Speed, dtype: float64
0.35793329506082994
-0.2364366728440488
```

Concadenar Dataframes

```
data_1 = pd.DataFrame({1: ['A11', 'A21', 'A31', 'A41'],
                        2: ['A12', 'A22', 'A32', 'A42'],
                        3: ['A13', 'A23', 'A33', 'A43'],
                        4: ['A14', 'A24', 'A34', 'A44']},
                      index=[1, 2, 3, 4])
```

```
data_2 = pd.DataFrame({1: ['A51', 'A61', 'A71', 'A81'],
                        2: ['A52', 'A62', 'A72', 'A82'],
                        3: ['A53', 'A63', 'A73', 'A83'],
                        4: ['A54', 'A64', 'A74', 'A84']},
                      index=[5, 6, 7, 8])
```

```
pd.concat([data_1,data_2]) # concatenamos los DataFrame
```

```
Out[105]:
   1  2  3  4
1  A11 A12 A13 A14
2  A21 A22 A23 A24
3  A31 A32 A33 A34
4  A41 A42 A43 A44
5  A51 A52 A53 A54
6  A61 A62 A63 A64
7  A71 A72 A73 A74
8  A81 A82 A83 A84
```

```
pd.concat([data_1,data_2], axis=1) #Por Default axis=0
```

```
Out[106]:
```

	1	2	3	4	1	2	3	4
1	A11	A12	A13	A14	NaN	NaN	NaN	NaN
2	A21	A22	A23	A24	NaN	NaN	NaN	NaN
3	A31	A32	A33	A34	NaN	NaN	NaN	NaN
4	A41	A42	A43	A44	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	A51	A52	A53	A54
6	NaN	NaN	NaN	NaN	A61	A62	A63	A64
7	NaN	NaN	NaN	NaN	A71	A72	A73	A74
8	NaN	NaN	NaN	NaN	A81	A82	A83	A84

ejemplo función y concatenado

```
def obtener_Z(df, columna):
```

```
    """Obtiene el valor Z de cada elemento de una columna determinada
```

```
    de un archivo seleccionado"""
```

```
    df2 = pd.DataFrame([])
```

```
    df2[f"Z_{columna}"] = (df[columna] - df[columna].mean()) / df[columna].std()
```

```
    return df2
```

```
obtener_Z(archivo_2, "Attack")
```

```
Out[108]:
```

	Z_Attack
Name	
Bulbasaur	-0.924328
Ivysaur	-0.523803
Venusaur	0.092390
VenusaurMega Venusaur	0.646964
Charmander	-0.831899
...	...
Diancie	0.646964
DiancieMega Diancie	2.495543
HoopahHoop Confined	0.955061
HoopahHoop Unbound	2.495543
Volcanion	0.955061

[800 rows x 1 columns]

```
obtener_Z(archivo_3, "Velocidad")
```

```
Out[109]:
```

	Z_Velocidad
0	-0.801002
1	-0.284837
2	0.403383
3	0.403383
4	-0.112782
..	...
795	-0.628947
796	1.435713
797	0.059273
798	0.403383
799	0.059273

[800 rows x 1 columns]

```
# Sacamos los valores Z para Attack
```

```
Z_attack = obtener_Z(archivo_2, "Attack")
```

```
Z_velocidad=obtener_Z(archivo_2, "Speed")
```

```
pd.concat([Z_attack, Z_velocidad ], axis=1)
```

```
Out[112]:
```

	Z_Attack	Z_Speed
Name		
Bulbasaur	-0.924328	-0.801002
Ivysaur	-0.523803	-0.284837
Venusaur	0.092390	0.403383
VenusaurMega Venusaur	0.646964	0.403383
Charmander	-0.831899	-0.112782
...	...	...
Diancie	0.646964	-0.628947
DiancieMega Diancie	2.495543	1.435713
Hoopahoop Confined	0.955061	0.059273
Hoopahoop Unbound	2.495543	0.403383
Volcanion	0.955061	0.059273

[800 rows x 2 columns]

Intervalos de Clase

```
#Para transformar las variables continuas en discretas,
```

```
#las ponemos en contenedores.
```

```
# Con Cut le indicamos cuanto contenedores queremos
```

```
archivo_5=pd.cut(archivo_3["Velocidad"], bins=15)
```

Index	Velocidad
0	(40.0, 51.667]
1	(51.667, 63.333]
2	(75.0, 86.667]
3	(75.0, 86.667]
4	(63.333, 75.0]
5	(75.0, 86.667]
6	(98.333, 110.0]
7	(98.333, 110.0]
8	(98.333, 110.0]
9	(40.0, 51.667]
10	(51.667, 63.333]

```
# qcut se Basa en quantiles
```

```
archivo_3["Velocidad_IC"] = pd.qcut(archivo_3["Velocidad"],[0,0.25,0.5,0.75,1])
```

```
archivo_3["Velocidad_IC"]
```

```
Out[114]:
0      (4.999, 45.0]
1      (45.0, 65.0]
2      (65.0, 90.0]
3      (65.0, 90.0]
4      (45.0, 65.0]
...
795     (45.0, 65.0]
796     (90.0, 180.0]
797     (65.0, 90.0]
798     (65.0, 90.0]
799     (65.0, 90.0]
Name: Velocidad_IC, Length: 800, dtype: category
Categories (4, interval[float64, right]): [(4.999, 45.0] < (45.0, 65.0] <
(65.0, 90.0] <
(90.0, 180.0]]
```

Dummy Variables

```
pd.get_dummies(data = archivo_2, columns=["Legendary"], prefix="Leg_Dummy",
drop_first=True )
```

```
pd.get_dummies(data = archivo_2, columns=["HP"], drop_first=True )
```

```
pd.get_dummies(data = archivo_2, columns=["Type 1"], drop_first=True)
```

```
Out[116]:
#      Type 2  ...  Type 1_Steel  Type 1_Water
Name
Bulbasaur    1  Poison  ...        False        False
Ivysaur      2  Poison  ...        False        False
Venusaur     3  Poison  ...        False        False
VenusaurMega Venusaur  3  Poison  ...        False        False
Charmander   4  Ninguno  ...        False        False
...
Diancie      719  Fairy  ...        False        False
DiancieMega Diancie  719  Fairy  ...        False        False
HoopaHoopa Confined  720  Ghost  ...        False        False
HoopaHoopa Unbound  720  Dark   ...        False        False
Volcanion    721  Water  ...        False        False
[800 rows x 32 columns]
```

Otras funciones relacionadas al Texto

```
archivo_2.reset_index() #Volver al Índice Orinigal
```

Cambiar los nombres de Columna

```
archivo_2.rename(columns = {'Type 1':'Poder 1', "Type 2": "Poder 2", "HP": "Salud"},
inplace=True)
```

```
archivo_2.columns
```

```
Out[117]:
```

```
Index(['#', 'Poder 1', 'Poder 2', 'Total', 'Salud', 'Attack', 'Defense',
      'Sp. Atk', 'Sp. Def', 'Speed', 'Generation', 'Legendary',
```

```
'Poder de Ataque', 'Poder de Defensa', 'Z_HP', 'Reacción'],
```

```
dtype='object')
```

```
archivo_2.columns = [x.lower() for x in archivo_2.columns]
```

Name	#	poder 1	poder 2	total	salu
Bulbasaur	1	Grass	Poison	318	45

```
archivo_3.columns=[x.upper()for x in archivo_3.columns]
```

Index	NOMBRE	VELOCIDAD	GENERACIÓN	VELOCIDAD_IC
0	Bulbasaur	45	1	(4.999, 45.0]

```
archivo_2.columns.values[0] ="Poder tipo 1"
```

```
archivo_2.columns.values[1] ="Poder tipo 2"
```

Name	Poder tipo 1	Poder tipo 2	poder 2	total	salu
Bulbasaur	1	Grass	Poison	318	45
Ivysaur	2	Grass	Poison	405	60
Venusaur	3	Grass	Poison	525	80
VenusaurMega Venusaur	3	Grass	Poison	625	80
Charmander	4	Fire	Ninguno	309	39
Charmeleon	5	Fire	Ninguno	405	58
Charizard	6	Fire	Flying	534	78
CharizardMega Charizard X	6	Fire	Dragon	634	78

Exportar Archivos obtenidos:

#Exportando a CSV

```
archivo_2.to_csv('Pokemon.csv',index=False)
```

#Exportando a Excel

```
archivo_2.to_excel("Pokemon.xlsx", index=True)
```

Armado de gráficos con Matplotlib

Grafico de barras verticales

```
import matplotlib.pyplot as plt
```

```
## Declaramos valores para el eje x
```

```
eje_x = ['Matemática', 'Biología', 'Economía', 'Ingeniería']
```

```
## Declaramos valores para el eje y
```

```
eje_y = [75,83,108,120]
```

```
## Creamos Gráfica
```

```
plt.bar(eje_x, eje_y)
```

```
## Leyenda en el eje y
```

```
plt.ylabel('Cantidad de estudiantes')
```

```
## Leyenda en el eje x
```

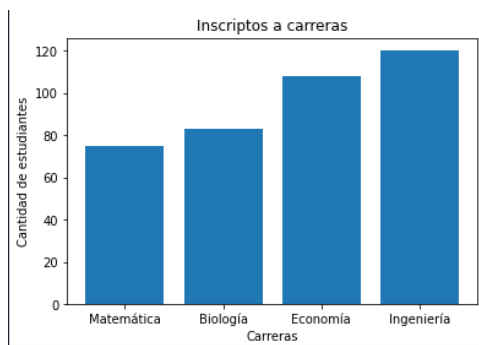
```
plt.xlabel('Carreras')
```

```
## Título
```

```
plt.title('Inscriptos a carreras')
```

```
## Mostramos el gráfico
```

```
plt.show()
```



```
## Grafica de barras horizontales
```

```
## Declaramos valores para el eje y, en este caso son categorias
```

```
eje_x = ['Programacion', 'Física', 'Matematicas', 'Química']
```

```
## Declaramos valores para el eje x, ahora son los valores
```

```
eje_y = [76,31,45,57]
```

```
## Creamos Gráfica
```

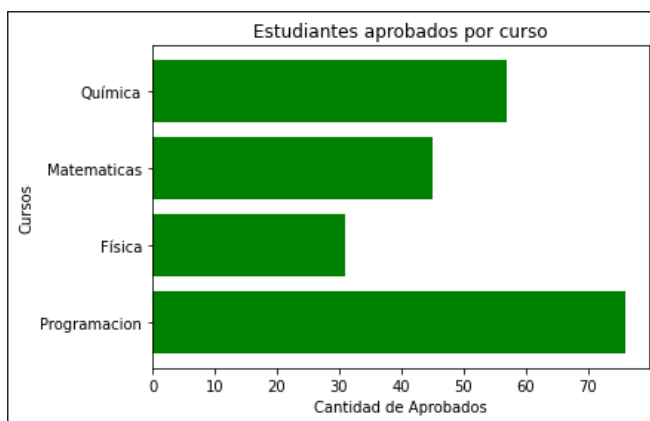
```
plt.barh(eje_x, eje_y, color="green")
```

```
plt.ylabel('Cursos')
```

```
plt.xlabel('Cantidad de Aprobados')
```

```
plt.title('Estudiantes aprobados por curso')
```

```
plt.show()
```



```
## Gráfico de barras agrupadas
```

```
serie_1 = [1089, 1287, 842, 957, 1485]
```

```
serie_2 = [1126, 1453, 735, 1078, 1512]
```

```
numero_de_grupos = len(serie_1)
```

```
indice_barras = np.arange(numero_de_grupos)
```

```
ancho_barras = 0.40
```

```

plt.bar(indice_barras, serie_1, width=ancho_barras, label='Hombres')

plt.bar(indice_barras + ancho_barras, serie_2, width=ancho_barras, label='Mujeres')

plt.legend(loc='best')

## Se colocan los indicadores en el eje x

plt.xticks(indice_barras + ancho_barras, ('Norte', 'Este', 'Sur', 'Oeste', 'Centro'))

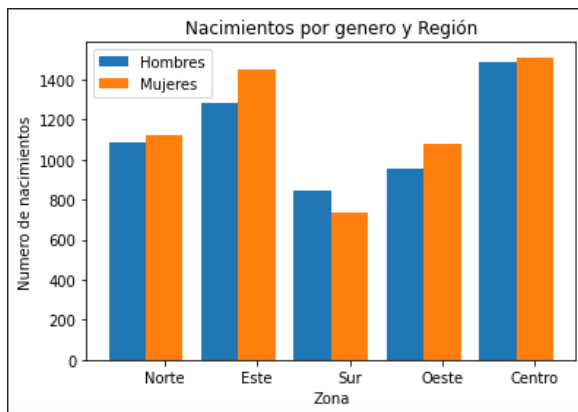
plt.ylabel('Numero de nacimientos')

plt.xlabel('Zona')

plt.title('Nacimientos por genero y Región')

plt.show()

```



Grafica de Barras apiladas

```

grupos = ['Grupo 1', 'Grupo 2', 'Grupo 3', 'Grupo 4']

```

```

Medicamento_A = [286,450,304,808]

```

```

Medicamento_B= [325,732,370,200]

```

```

indice = np.arange(len(grupos))

```

Se crean las primeras barras

```

plt.bar(indice, Medicamento_A, label='Medicamento A')

```

Se crean las segundas barras y se apilan sobre las primeras

```
plt.bar(indice, Medicamento_B, label='Medicamento B', bottom=Medicamento_A)
```

```
plt.xticks(indice, grupos)
```

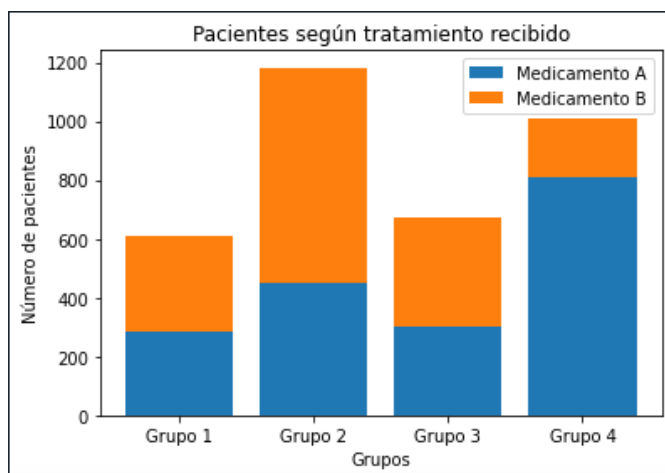
```
plt.ylabel("Número de pacientes")
```

```
plt.xlabel("Grupos")
```

```
plt.title('Pacientes según tratamiento recibido')
```

```
plt.legend()
```

```
plt.show()
```



Herramientas para Simulaciones

#Se puede generar y trabajar con distribuciones de probabilidad.

Distribución Normal

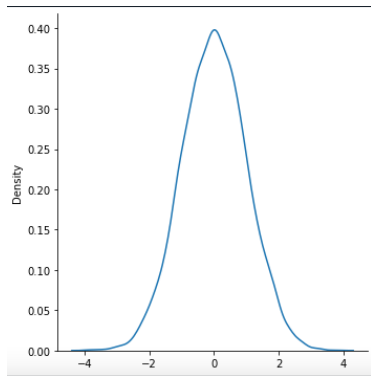
mu = 0 #definimos la media

sigma = 1 #definimos el desvío

#ahora, generamos 1000 números con media 0 y desvío 1

```
dist_norm = np.random.normal(mu, sigma, 1000)
```

```
#graficamos una normal 0,1 con 10000 valores simulados  
sb.displot(np.random.normal(mu, sigma, 10000), kind = 'kde')  
plt.show()
```



```
#iteramos para una lista de n cantidad de variables
```

```
n_muestras = [10, 1000, 10000]
```

```
mu = 0
```

```
sigma = 1
```

```
for n in n_muestras:
```

```
    norm_dist = np.random.normal(mu, sigma, n)
```

```
    print("Para un n de: {} Media: {} Varianza: {}".format(n, norm_dist.mean(), norm_dist.std()))
```

```
    sb.distplot(norm_dist, hist = False, kde = True,
```

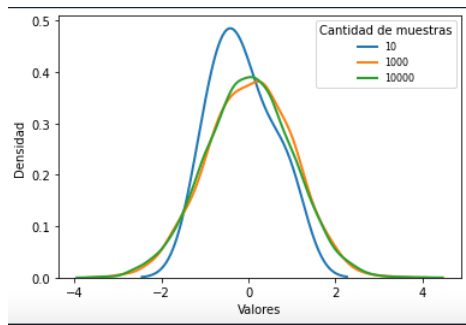
```
                kde_kws = {'linewidth': 2}, label = n)
```

```
plt.legend(prop={'size': 8}, title = 'Cantidad de muestras')
```

```
plt.xlabel('Valores')
```

```
plt.ylabel('Densidad')
```

```
plt.show()
```



#Tomamos una lista con diferentes valores de mu y sigma

```
parametros = [(1,2.4),(4,1.5),(5,5)]
```

```
for mu, sigma in parametros:
```

```
    sb.distplot(np.random.normal(mu, sigma, 1000), hist = False, kde = True,
```

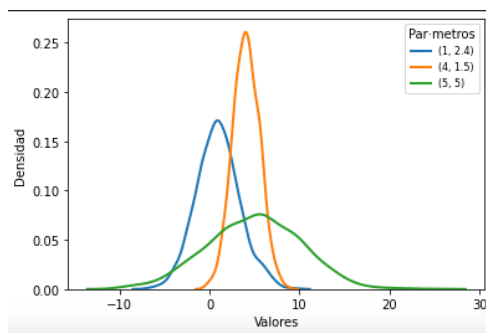
```
               kde_kws = {'linewidth': 2}, label = (mu,sigma))
```

```
plt.legend(prop={'size': 8}, title = 'Par·metros')
```

```
plt.xlabel('Valores')
```

```
plt.ylabel('Densidad')
```

```
plt.show()
```



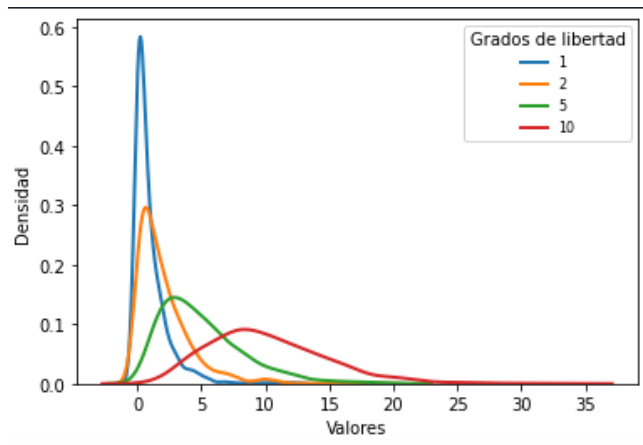
Distribución Chi Cuadrado

#Simulamos 1000 valores de una chi cuadrado para diferentes grados de libertad

```
for gl in [1,2,5,10]:
```

```
sb.distplot(np.random.chisquare(gl,1000), hist = False, kde = True,  
            kde_kws = {'linewidth': 2}, label = gl)
```

```
plt.legend(prop={'size': 8}, title = 'Grados de libertad')  
plt.xlabel('Valores')  
plt.ylabel('Densidad')  
plt.show()
```



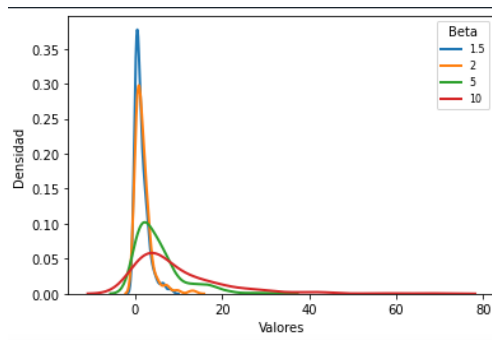
Distribución Exponencial

#Simulamos una distribución exponencial para diferentes valores de beta

```
for beta in [1.5,2,5,10]:
```

```
    sb.distplot(np.random.exponential(beta,200), hist = False, kde = True,  
                kde_kws = {'linewidth': 2}, label = beta)
```

```
plt.legend(prop={'size': 8}, title = 'Beta')  
plt.xlabel('Valores')  
plt.ylabel('Densidad')  
plt.show()
```



Análisis de Distribuciones

Tests de Media y Varianza

#Paso 1- Simulamos 2 poblaciones de 100000 valores cada una

```
poblacion_1 = np.random.normal(1.09, 2, 100000).tolist()
```

```
poblacion_2 = np.random.normal(1, 2, 100000).tolist()
```

#Paso 2- Importamos las librerías a utilizar

```
from scipy import stats
```

```
from random import sample
```

#Paso-3 generamos una submuestra de cada población aleatoria de 5000 valores cada una

```
submuestra_1 = sample(poblacion_1, 5000)
```

```
submuestra_2 = sample(poblacion_2, 5000)
```

#Paso-4 evaluamos las medias (H0=las medias son iguales;H1=las medias son distintas)

```
t = stats.ttest_ind(poblacion_1, poblacion_2, equal_var = False, nan_policy = 'omit', alternative = 'two-sided')
```

```
tt_stat, pvalue = t[0], t[1]
```

```
pvalue
```

```
Out[131]: 4.5404729493476826e-21
```

#esto es sensible al tamaño de la muestra, por eso usamos las muestras grandes

#si $pvalue \leq \alpha$ ---rechazo H_0 ; si $pvalue > \alpha$ No rechazo H_0

#Armamos una función que evalúa el test de medias

```
def test_medias(submuestra_1, submuestra_2, alpha, regla_2t, equal_var):
```

```
    tt = stats.ttest_ind(submuestra_1, submuestra_2, equal_var = equal_var, nan_policy='omit',
alternative='two-sided')
```

```
    tt_stat, pvalue = tt[0], tt[1]
```

```
    if regla_2t:
```

```
        if tt_stat > 2:
```

```
            print("Las muestras no provienen de la misma distribución; RECHAZO  $H_0$ ")
```

```
        else:
```

```
            print("Las muestras provienen de la misma distribución, NO RECHAZO  $H_0$ ")
```

```
    else:
```

```
        if pvalue < alpha:
```

```
            print("Las muestras no provienen de la misma distribución")
```

```
        else:
```

```
            print("Las muestras provienen de la misma distribución")
```

```
test_medias(submuestra_1, submuestra_2, alpha=0.05, regla_2t=True, equal_var=True)
```

Las muestras provienen de la misma distribución, NO RECHAZO H_0

#iterando:

```
pob_1 = np.random.normal(1.09, 2, 100000).tolist()
```

```
pob_2 = np.random.normal(1, 2, 100000).tolist()
```

```
aux = [test_medias(sample(pob_1, n_muestras), sample(pob_2, n_muestras), alpha=0.05,  
regla_2t=True, equal_var =True) for n_muestras in range(100, len(pob_1)+1,10000)]
```

```
alpha = 0.05
```

```
n_muestras = 0
```

```
iters = 0
```

```
print('Iter | # samples')
```

```
print('-----')
```

```
while True:
```

```
    n_muestras += 500
```

```
    iters += 1
```

```
    print(iters, ' | ', n_muestras*10)
```

```
    sub_1 = sample(pob_1, n_muestras*10)
```

```
    sub_2 = sample(pob_2, n_muestras*10)
```

```
    tt = stats.ttest_ind(sub_1, sub_2, equal_var = True, nan_policy='omit', alternative='two-  
sided')
```

```
    tt_stat, pvalue = tt[0], tt[1]
```

```
    if pvalue < alpha:
```

```
        break
```

```
print(f'Se necesitaron {n_muestras*10} (iteraciones: {iters}) muestras para poder determinar  
con un alpha de 0.05 \
```

que las muestras NO provienen de las misma población.')

Las muestras provienen de la misma distribución, NO RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Las muestras no provienen de la misma distribución; RECHAZO H0

Iter | # samples

1 | 5000

Se necesitaron 5000 (iteraciones: 1) muestras para poder determinar con un alpha de 0.05 que las muestras NO provienen de las misma población.